

COMPUTER ORGANIZATION AND ARCHITECTURE

UNIT – V SHORT NOTES

Overview of Pipelining Technique & Processor Architecture

1. Parallel Processing & Flynn's Classification
2. Pipelining – Basics & Examples
3. Arithmetic Pipeline
4. Instruction Pipeline
5. RISC Pipeline – Delayed Load & Branch
6. Pipeline Hazards – Data, Structural, Branch
7. Branch Prediction
8. RISC vs CISC

1. PARALLEL PROCESSING

Definition

Parallel processing is a class of techniques that enables a computer system to perform **multiple data-processing tasks simultaneously** to increase computational speed and throughput (amount of work done in a given time).

Need for Parallel Processing

- To increase the **throughput** (amount of processing in a given time).
- To **speed up** large computations that serial computers cannot handle efficiently.
- Achieved by having **multiple functional units** operating simultaneously on different data.

Flynn's Classification (Flynn's Taxonomy)

Proposed by **M. J. Flynn**. Classifies computers based on the number of **instruction streams** and **data streams** processed simultaneously.

Category	Instruction Stream	Data Stream	Example
SISD	Single	Single	Von Neumann / Normal PC
SIMD	Single	Multiple	Pipelined Processors
MISD	Multiple	Single	Theoretical / Rare
MIMD	Multiple	Multiple	Multiprocessors / Superscalar

SISD – Single Instruction, Single Data

- Uniprocessor (one CPU) executes one instruction on one data at a time.
- Instructions are processed **sequentially**.
- Most conventional computers use this. Example: Von-Neumann computers.

SIMD – Single Instruction, Multiple Data

- One control unit supervises **many processing elements (PEs)**.
- All PEs receive the **same instruction** but work on **different data**.
- Example: Pipelined Processors.

MISD – Multiple Instruction, Single Data

- Multiple processors, each with different instructions, work on **one data stream**.
- Output of one processor becomes input of the next (macro pipe).
- Considered **impractical** in many applications.

MIMD – Multiple Instruction, Multiple Data

- Multiple processors each execute their own instructions on their own data.
- Most modern multiprocessor systems use this.
- Example: Superscalar Processors.

Advantages of Parallel over Serial Computing

- Saves time and money – many resources working together reduce execution time.
- Can solve problems that are **too large for serial computers**.
- Can use non-local resources when local ones are limited.

Disadvantages of Parallel over Serial Computing

- Increases hardware cost (more components needed).
- Multicore architectures consume **more power**.
- Difficult to design and implement.
- Overhead due to synchronization (coordination between processors) and data transfers.

2. PIPELINING

Definition

Pipelining is a technique of **decomposing a sequential process into sub-operations**, where each sub-operation is executed in a dedicated segment (stage) that operates **concurrently** (at the same time) with all other segments. It works like an industrial assembly line.

Key Points

- Several computations can be in progress in **distinct segments simultaneously**.
- The name 'pipeline' comes from the analogy of water flowing through connected pipes.
- Pipelining increases **throughput**, not the speed of individual instructions.

Principles of Pipelining

- Each task is divided into a number of successive sub-tasks.
- A dedicated pipeline stage is associated with each sub-task.
- Each stage takes the **same amount of time** (one clock cycle).
- Output of one stage is given as input to the next stage.
- Pipeline is clocked **synchronously** (all stages tick at the same time).

Example – Laundry Analogy

3 stages: Wash (30 min) → Dry (30 min) → Fold (30 min). 4 loads of clothes.

- **Without pipelining (Sequential):** $4 \times 3 \times 30 = 360 \text{ min} = 6 \text{ hours}$
- **With pipelining:** $30 + 30 + (4 \times 30) = 180 \text{ min} = 3 \text{ hours} \rightarrow \text{Speedup} = 2\times$

Serial Execution vs Pipelining – Speedup Formula

For a **k-segment pipeline** executing **n tasks**, with clock cycle time **t_p** :

$$\text{Total pipeline time} = [k + (n - 1)] \times t_p$$

For a non-pipelined unit taking time t_n per task:

$$\text{Total serial time} = n \times t_n$$

$$\text{Speedup (S)} = n \times t_n / [k + (n-1)] \times t_p$$

When n is very large: $S \approx t_n / t_p$. If $t_n = k \times t_p$, then $S = k$.

→ **Maximum speedup = k (number of pipeline stages)**

3. ARITHMETIC PIPELINE

Arithmetic Pipeline divides an arithmetic problem into sub-problems executed in separate pipeline segments. Mostly used in **high-speed computers** for floating-point operations, fixed-point multiplication, etc.

Example – Floating-Point Addition Pipeline

Given two floating-point numbers:

$$X = 0.9504 \times 10^3 \quad Y = 0.8200 \times 10^2$$

The operation is divided into **4 segments**:

Segment	Operation	Detail
Segment 1	Compare Exponents	Subtract exponents to find difference: $3 - 2 = 1$
Segment 2	Align Mantissas	Shift smaller mantissa right by 1: Y becomes 0.08200×10^3
Segment 3	Add Mantissas	$Z = 0.9504 + 0.0820 = 1.0324 \times 10^3$
Segment 4	Normalize Result	Shift to get: $Z = 0.10324 \times 10^4$ ■

4. INSTRUCTION PIPELINE

An **instruction pipeline** overlaps the fetch, decode, and execute phases of multiple instructions to increase throughput.

Steps to Process an Instruction

- **Step 1 – Fetch Instruction (FI):** Read instruction from memory
- **Step 2 – Decode Instruction (DA):** Identify the operation and calculate effective address
- **Step 3 – Fetch Operand (FO):** Get required data from memory or registers
- **Step 4 – Execute (EX):** Perform the actual operation

Four-Segment Instruction Pipeline (FI → DA → FO → EX)

- While instruction 1 is being executed (EX), instruction 2 is fetching its operand (FO), instruction 3 is being decoded (DA), and instruction 4 is being fetched (FI).
- Thus, up to **4 instructions can be in progress simultaneously**.

Effect of Branch Instructions on Pipeline

- If a branch instruction is encountered, the pipeline may have fetched the **wrong next instruction**.
- The **incorrectly fetched instructions must be discarded** (flushed from the pipeline).
- The pipeline **restarts from the new branch address** stored in the Program Counter (PC).
- An **interrupt request** also causes the pipeline to empty and restart from a new address.

5. RISC PIPELINE

RISC (Reduced Instruction Set Computer) processors are designed to use an **efficient instruction pipeline**. Their simple, fixed-length instructions allow decoding and register selection to happen simultaneously.

Three-Segment RISC Instruction Pipeline

Segment	Name	Operation
I	Instruction Fetch	Fetch instruction from program memory
A	ALU Operation	Decode instruction + perform ALU operation / calculate effective address / compute branch
E	Execute	Write ALU result to register file, OR access data memory for load/store, OR update PC

Delayed Load

A **data conflict** (hazard) occurs when an instruction tries to use data from a LOAD that has not yet completed.

- **Problem:** LOAD R2 places data into R2 at the end of clock cycle 4. But ADD R1+R2 tries to use R2 in clock cycle 4 itself → Wrong value is used!
- **Solution:** The compiler inserts a **NOP (No-Operation)** instruction after the LOAD. NOP wastes one clock cycle, giving the LOAD time to complete.
- **Advantage:** Data dependency is handled by the **compiler**, not hardware → simpler hardware design.
- This approach is called **Delayed Load**.

Delayed Branch

Branch instructions cause delays because the processor has already fetched wrong instructions while the branch was being processed.

- **Delayed Branch:** The compiler **rearranges instructions** so that useful instructions are placed after the branch (in the 'delay slots'), making the pipeline work efficiently.
- If no useful instruction is available, the compiler inserts **NOP instructions** in the delay slots.
- This relies on the **compiler** to manage the pipeline, not hardware.

6. PIPELINE HAZARDS

Hazard: Any situation or condition that causes the pipeline to **stall** (pause), preventing the next instruction from executing in its designated clock cycle. When an instruction stalls, all later instructions also stall.

Types of Hazards

- **1. Data Hazards** – caused by data dependency between instructions
- **2. Structural Hazards** – caused by resource conflicts (hardware not available)
- **3. Control/Branch Hazards** – caused by branch instructions

A. Data Hazards

Occurs when an instruction needs data that is **not yet ready** because a previous instruction hasn't finished writing it.

Three Types of Data Hazards

Type	Also Called	Description	Example
RAW (Read After Write)	True Dependency / Flow Dependency	Instruction i reads a value before instruction i has finished writing it	I1: R2 ← R1+R3 I2: R4 ← R2+R3 (I2 needs R2 which I1 hasn't written yet)

WAR (Write After Read)	Anti-Dependency / Instruction j tries to write a location before instruction i has read it False Dependency		I1: $R4 \leftarrow R1 + R3$ I2: $R3 \leftarrow R1 + R2$ (I2 writes R3 before I1 reads it)
WAW (Write After Write)	Output Dependency Instruction j tries to write before instruction i has written to the same location		I1: $R2 \leftarrow R1 + R2$ I2: $R2 \leftarrow R4 + R7$ (Both write to R2 – order must be preserved)

Resolving Data Hazards

Method 1: Pipeline Bubbling (NOP Insertion)

- Control logic detects the hazard and inserts a **NOP (No-Operation)** instruction.
- NOP does nothing but occupies a clock cycle, giving the previous instruction time to complete.
- Advantage:** Simpler hardware. **Disadvantage:** Larger code size (more instructions).
- Flushing the pipeline:** If NOPs equal the number of pipeline stages, all instructions are cleared — called flushing.

Method 2: Data Forwarding (Operand Forwarding)

- A **hardware-based** solution. Once the ALU (Arithmetic Logic Unit) completes an operation, its result is **directly forwarded** (fed back) to the next instruction's input without waiting to be written to a register first.
- Reduces or **eliminates the stall** caused by data hazards.
- Forwarding path:** Extra hardware connection from ALU output back to ALU input of the next stage.

B. Structural Hazards

Occurs when **two instructions require the same hardware resource** (e.g., memory, register file) at the same time.

Common Cause

- One instruction accesses memory during Execute/Write stage while another instruction is being **fetched from the same memory**.
- If instruction and data share the same cache, only one can proceed — the other is delayed.

Resolution

- Separate instruction and data caches** – Most modern processors use two separate caches to eliminate this conflict.
- Stalling (Bubbling)** – Insert a bubble into the pipeline for one clock cycle to allow the conflict to resolve.
- Increasing hardware resources** – e.g., register file with multiple input/output ports.

Why designers allow structural hazards?

- To **reduce cost** – Supporting separate instruction and data memory access every cycle requires twice as much memory.
- To **reduce latency** – Fewer pipeline registers means less overhead.

C. Branch (Control / Instruction) Hazards

Occurs when the processor encounters a **branch instruction**. The processor doesn't know which instruction to fetch next (normal sequence or branch target) until the branch is resolved.

Unconditional Branch

- Branch target is always taken. The processor still fetches the wrong next instruction before discovering the branch.
- Branch Penalty:** Time lost due to fetching wrong instructions. If branch address is computed in the **decode stage** → penalty = 1 cycle. If computed in the **execute stage** → penalty = 2 cycles.

Conditional Branch

- Even more complex – branch decision depends on a **condition** that may not be evaluated yet.
- The processor cannot decide to branch until the condition is computed.

Instruction Queue and Prefetching

- Many processors use a dedicated **fetch unit** that fetches instructions ahead of time and places them in an **instruction queue**.
- A **dispatch unit** takes instructions from the queue and sends them to the execution unit.
- Beneficial during cache misses – dispatch unit keeps sending instructions from the queue even when fetching is stalled.

Controlling Branch Hazards – Software Solutions

- Branch spreading, Loop unrolling, Trace scheduling
- **Scheduling the branch delay slot:** Place useful instructions in the slots after a branch.
- **Static branch prediction:** Predict taken or not taken at compile time; insert NOP if wrong.
- **Pipeline Bubbling:** Insert NOP – guaranteed to increase latency.

Controlling Branch Hazards – Hardware Solutions

- **Halting the pipeline:** Stop fetching new instructions until the branch is resolved. Always causes delay.
- **Branch Prediction:** Hardware guesses the branch direction, continues fetching. If wrong, flushes and restarts.
- **Prefetching multiple paths:** Fetch instructions from BOTH the branch target and sequential path simultaneously; select the correct one after the condition is evaluated.

7. BRANCH PREDICTION

Branch Prediction is a technique used to **reduce the branch penalty** associated with branch instructions by predicting (guessing) whether a branch will be **taken** or **not taken** before the condition is actually evaluated.

A. Static Branch Prediction

Prediction is made **before the program runs** (at compile time) based only on the branch instruction itself. Does **not use runtime history**.

1. Predict Not Taken

- Assumes the branch will **NOT** be taken and continues fetching instructions sequentially.
- Instructions are executed **speculatively** (before we are sure they should be).
- If prediction is wrong: fetched instructions are **purged** and the correct instruction is fetched from the branch target.
- Registers/memory are **not updated** until the prediction is confirmed correct.

2. Predict Taken

- Assumes the branch **WILL** be taken. Fetch from the branch target address.
- Always incurs at least **one clock cycle penalty** while the branch target address is computed.
- **Predict Not Taken is slightly better** in performance; Predict Taken is more complex to implement.

B. Dynamic Branch Prediction

Prediction is made at **runtime** using the **past history** of that branch instruction. Uses a **Branch Prediction Buffer (Branch History Table)** – a small fast memory storing history of previous branch outcomes.

2-State Algorithm (1-bit predictor)

- Two states: **LT (Likely Taken)** and **LNT (Likely Not Taken)**.
- If branch is taken → move to LT state. If not taken → move to LNT state.

- Uses **1 bit** of history per branch instruction.
- Problem: Changes prediction after just one wrong outcome → poor for loops.

4-State Algorithm (2-bit predictor)

- Four states: **ST (Strongly Taken)**, **LT (Likely Taken)**, **LNT (Likely Not Taken)**, **SNT (Strongly Not Taken)**.
- Prediction must be wrong **twice in a row** before the prediction changes.
- Better performance than 2-state – more stable predictions, especially for loops.
- Uses **2 bits** of history per branch instruction.

8. RISC AND CISC

RISC – Reduced Instruction Set Computer

Uses a **small set of simple, fixed-length instructions**, each executing in **one clock cycle**. Emphasizes software (compiler) to do complex tasks.

Characteristics of RISC

- Simple instructions → **simple decoding**.
- Each instruction is **one word in size**.
- Each instruction takes **one clock cycle** to execute.
- Large number of **general-purpose registers**.
- Simple addressing modes. Fewer data types.
- **Only LOAD and STORE instructions** access memory.
- Pipelining is **easy to implement**.
- Examples: SPARC, PowerPC, ARM.

Advantages of RISC

- **Faster execution** – simple instructions execute in one cycle.
- **Lower power consumption** – ideal for mobile devices.
- **Simpler hardware design** – easier to manufacture.

Disadvantages of RISC

- **More instructions needed** to do complex tasks → larger code.
- **More memory usage** to store the extra instructions.
- **Complex compiler** – compiler must do more work.

CISC – Complex Instruction Set Computer

Uses a **large set of complex, variable-length instructions**, where one instruction may do multiple operations (load + compute + store). Emphasizes hardware.

Characteristics of CISC

- Complex instructions → **complex decoding**.
- Instructions can be **larger than one word**.
- Instructions may take **multiple clock cycles**.
- Fewer general-purpose registers (operations done in memory).
- Complex addressing modes. More data types.
- Any instruction can access memory.
- Pipelining is **difficult to implement**.
- Examples: Intel x86, AMD, 8085.

Advantages of CISC

- **Reduced code size** – complex instructions do more work per instruction.
- **More memory efficient** – fewer instructions needed.
- **Simpler compiler** – less work for the compiler.

Disadvantages of CISC

- **Slower execution** – complex instructions take more cycles.
- **More complex hardware** – harder to design.
- **Higher power consumption.**

RISC vs CISC – Key Comparison Table

Feature	RISC	CISC
Instruction Set	Small, simple	Large, complex
Instruction Size	Fixed (one word)	Variable
Execution Time	One clock cycle per instruction	Multiple clock cycles possible
Memory Access	Only LOAD and STORE	Any instruction can access memory
Registers	Many general-purpose	Few general-purpose
Emphasis	Software (compiler)	Hardware
Pipelining	Easy to implement	Difficult to implement
Power Consumption	Lower	Higher
Code Size	Larger (more instructions)	Smaller (fewer instructions)
Compiler Complexity	More complex	Simpler
Examples	SPARC, PowerPC, ARM	Intel x86, AMD, 8085

RISC vs CISC Example – Multiplying Two Numbers

Task: Multiply number at memory location 2:3 with number at 5:2, store result back at 2:3.

CISC Approach

Uses a single complex instruction: **MULT 2:3, 5:2**

- One instruction does everything: load → multiply → store.
- Less code to write, less RAM for instructions.
- But hardware must be complex to support this.

RISC Approach

Uses multiple simple instructions:

- **LOAD A, 2:3** → Load first number into register A
- **LOAD B, 5:2** → Load second number into register B
- **PROD A, B** → Multiply A and B, store in A
- **STORE 2:3, A** → Store result back to memory
- More lines of code, but each executes in one clock cycle → same total time.
- Pipelining is possible because all instructions take the same time (one cycle).
- Fewer transistors needed → room for more registers.